An Artificial Neural Network Representation of the SABR Stochastic Volatility Model

William A. McGhee*
NatWest Markets
250 Bishopsgate, London EC2M 4AA

21st November 2018

Abstract

In this article, the Universal Approximation Theorem of Artificial Neural Networks (ANNs) is applied to the SABR stochastic volatility model in order to construct highly efficient representations. Initially, the SABR approximation of Hagan et al. [2002] is considered, then a more accurate integration scheme of McGhee [2011] as well as a two factor finite difference scheme. The resulting ANN calculates 10,000 times faster than the finite difference scheme whilst maintaining a high degree of accuracy. As a result, the ANN dispenses with the need for the commonly used SABR Approximation.

Keywords: Stochastic Volatility, SABR model, SABR Approximation, SABR Integration Scheme, Artificial Neural Network, Universal Approximation Theorem.

1 Introduction

Ever since the introduction of the SABR Model (Hagan et al. [2002]) and the widely used approximation presented therein, there has been a fuller understanding of the weaknesses in applying it across a wide parameter space and time domain. A great deal of effort has been expended on producing more and more refined and efficient representations (see, for example, Islah [2009], McGhee [2011], Chen et al. [2012], Antonov and Spector [2013]). The original approximation and its extensions are within constant use across a large number of institutions within financial markets, covering a range of asset classes.

*email: William.McGhee@natwestmarkets.com

The underlying model can be fully represented using a two factor finite difference scheme but this in itself presents a number of implementation challenges, in addition to being impractically non-performant. As such, there has been a reliance on increasingly sophisticated approximations.

An alternative, general approach using Artificial Neural Networks (ANNs) is presented in this article for the SABR stochastic volatility model, using the $\beta = 1$ case as a working example, with extensions to other models and calculations left for a future article.

There is widespread use of Artificial Neural Networks across many industries and subject areas, where they are used to extract modellable features from large data sets. The ability of an Artificial Neural Network to build complex functional relationships is due to the property, as expressed via the Universal Approximation Theorem (Cybenko [1989]), that it is able to represent a general function of an n-dimensional real variable to a given degree of accuracy using one hidden layer and a sigmoid activation function (with extensions to more general functions as shown in Hornik [1991]). The number of nodes in the hidden layer, although finite, is unknown as are the weights of the network which needs to be calculated via a "learning" algorithm.

For many applications, more elaborate neural network configurations are used, including deep networks, recurrent networks, long short term memory networks and convolutional networks. However, in this article the power of relatively simple networks with one hidden layer will be shown to encapsulate a stochastic volatility model for the chosen parameter range.

A number of articles are emerging which also consider the principles explored here. The recent report by Brostrom and Kristiansson [2018] looks at the effectiveness of ANNs to represent vanilla and rainbow options within a Value-at-Risk context. Further, Ferguson and Green [2018] have shown that basket options can be respresented via Artificial Neural Networks.

In the SABR case, the starting point will be the approximation scheme (Hagan et al. [2002]) which is in widespread use. Considering a representative domain space, a range of ANNs with differing numbers of neurons in the single hidden layer will be shown to be able to accurately represent the functional form of the approximation. This exercise demonstrates the ANN approach and is the foundation for applying the method to more computationally expensive, but more accurate, schemes.

As such, a numerical integration approach and that of a two factor finite difference scheme, are used to produce accurate implied volatilities corresponding to the underlying stochastic volatility process. Given the nature of these calculations though, the volatilities are time consuming to produce. The advantage of the method described in this paper, is that calibration of an ANN to these implementations can take place offline and computational resources can be deployed in advance to produce a model (the ANN), encapsulating SABR across a wide parameter space.

Furthermore, adopting the principle of "transfer learning", the ANN trained on the SABR Approximation can be used as the starting point for an ANN that represents the underlying SABR Process as calculated under a more computationally expensive scheme. The resulting ANN rep-

resentation is efficient in terms of calculation time, running 10,000 times faster than the finite difference scheme and 65 times faster than the integration scheme, whilst suffering from none of the weaknesses of the approximation. The extension to the $\beta < 1$ case simply requires sampling an extra dimension in the input space and will be left to a subsequent paper.

Even when the problems of the SABR approximation are removed, the model itself lacks the flexibility required to give adequate control over the wings of the smile (the low delta region relative to the high delta region). As noted in McGhee and Trabalzini [2014] the level of the smile wings is a function of the right hand tail of the mean integrated variance distribution i.e. high volatility paths. As such the wings can be controlled by introducing skew to the volatility process, either via CEV (Andreasen and Huge [2013]) or via stochastic volatility of volatility (McGhee [2013]). By their very nature, these are more complex models but the methodology employed here can be used to train these models offline also. This will also be investigated in a future article.

2 The Universal Approximation Theorem

A feed-forward neural network takes N inputs $\{x_p : p = 1, ..., N\}$ and through a series of weighted sums and the application of "activation functions", produces M outputs $\{y_q : q = 1, ..., M\}$. The Universal Approximation Theorem applied to a single hidden layer network and can be expressed as follows.

Theorem: Let $\phi(.)$ be a non-constant, bounded and monotonically increasing continuous function. Let $V_m \in \mathbb{R}^m$ be any compact subset of $\mathbb{R}^m$. The space of continuous functions on V_m is denoted by $C(V_m)$. Given $\epsilon > 0$ and any function $f \in C(V_m)$, there exists an integer N and constants $\nu_i, b_i \in \mathbb{R}$ and real vectors $\omega_i \in \mathbb{R}^m$, where $i = 1, ..., N$ such that

$$F(x) = \sum_{i=1}^{N} \nu_i \phi(\omega_i^T x + b_i) \tag{1}$$

as an approximate realisation of the function f, which is independent of $\phi(.)$. Specifically,

$$|F(x) - f(x)| < \epsilon \tag{2}$$

for all $x \in V_m$.

Proof See Cybenko [1989].

The primary focus of this paper will be the application of neural network to function representation. In keeping with the original result, a feedforward network with one hidden layer will be used, with differing numbers of neurons.

3 The SABR Model

The SABR model is now considered, in particular three implementations: the *SABR Approximation* (Hagan et al. [2002]), the *SABR Integration* (see McGhee [2011]) and a two factor finite difference implementation, the *SABR 2FPDE* (using Craig and Sneyd [1988]).

The SABR Model is a CEV Cox [1975] asset process with correlated lognormal stochastic volatility. A forward rate $F(t,T)$ to time T, observed at t, and the instantaneous volatility, $\sigma(t)$ follow the stochastic differential equations

$$dF(t,T) = \sigma(t)F(t,T)^{\beta}dW_F(t) \qquad \text{and} \qquad d\sigma(t) = \xi\sigma(t)dW_\sigma(t) \tag{3}$$

The Wiener processes $W_F(t)$ and $W_\sigma(t)$ $(0 \le t \le T)$ are correlated with $< dW_F(t), dW_\sigma(t) >= \rho dt$. The CEV function is determined by the exponent β with special cases being $\beta = 1$ where the forward process, conditional on the volatility path, is lognormal; and $\beta = 0$ where the forward process, conditional on the volatility path, is normal. The volatility of volatility parameter is ξ. Of interest is to determine the function that yields the implied volatility for a given strike and maturity i.e. $\sigma(K, T, \{\sigma(0), \xi, \rho\})$.

3.1 The SABR Approximation

The starting point will be the commonly used SABR Approximation (Hagan et al. [2002]) which for $\sigma_{approx}(K,T) \approx \sigma(K, T, \{\sigma(0), \xi, \rho\})$ takes the form

$$\sigma_{approx}(K,F) = \frac{\sigma(0)}{q(K,F)}\left(\frac{z}{x(z)}\right)\left[1 + \left[\frac{(1-\beta)^2}{24}\frac{\sigma(0)^2}{(FK)^{1-\beta}} + \frac{1}{4}\frac{\rho\beta\xi\sigma(0)}{(FK)^{(1-\beta)/2}} + \frac{2-3\rho^2}{24}\xi^2\right]T + ...\right] \tag{4}$$

for the implied volatility, where

$$q(K,F) = (FK)^{(1-\beta)/2}\left[1 + \frac{(1-\beta)^2}{24}\log^2\left(\frac{F}{K}\right) + \frac{(1-\beta)^4}{1920} + \log^4\left(\frac{F}{K}\right) + ...\right] \tag{5}$$

$$z = \frac{\xi}{\sigma(0)}(FK)^{(1-\beta)/2}\log\left(\frac{F}{K}\right) \tag{6}$$

and

$$x(z) = \log\left(\frac{\sqrt{1-2\rho z+z^2}+z-\rho}{1-\rho}\right) \tag{7}$$

The advantage of the original formula was the ease of implementation and computational effectiveness as well as being driven by an underlying model combining correlated stochastic volatility with the CEV. The formula, however, suffers from a number of weaknesses:

- exhibiting regions of negative implied probability for realistic parameter ranges;
- deviation from the underlying SDE, meaning that the SABR parameters used to calibrate to the market as determined using the approximation formula will differ from those used e.g. in Monte Carlo or finite difference schemes to calibrate to the same market data;
- the inability of the formula to calibrate to market smiles in the long dated space which are used to drive the pricing of contracts such as Power Reverse Dual Contracts (PRDCs)

To address these weaknesses a number of alternatives and improvements have been proposed including Islah [2009], McGhee [2011], Chen et al. [2012] and Antonov and Spector [2013].

3.2 The SABR Integration

In McGhee [2011] a SABR integration scheme is developed which integrates conditional option prices over the joint density of the terminal instantaneous volatility and the mean integrated variance. The cases of lognormal ($\beta = 1$), normal ($\beta = 0$) and CEV ($0.5 < \beta < 1$) asset processes are considered.

Here the case of $\beta = 1$ will be outlined. An equivalent derivation takes place for $\beta = 0$ where conditional lognormal prices are replaced with conditional normal prices. For $0.5 < \beta < 1$ a similar calculation is performed but a further approximation is required to handle the conditional CEV prices. See McGhee [2011] for more details.

For the lognormal ($\beta = 1$) asset process, correlated to an underlying stochastic volatility $\sigma(t)$,

$$d \ln F(t,T) = -\frac{1}{2}\sigma(t)^2 dt + \sigma(t) dW(t) \tag{8}$$

which can be integrated to yield

$$F(T,T) = F(0,T) \exp\left(-\frac{1}{2}\int_0^T \sigma(t)^2 dt + \int_0^T \sigma(t) dW(t) \right) \tag{9}$$

If the volatility process is driven by a Wiener process dW_σ correlated to the Wiener driver of the forward asset, dW, i.e. $< dW, dW_\sigma >= \rho dt$, then the asset Wiener process can be expressed as

$$dW(t) = \rho dW_\sigma(t) + \sqrt{1-\rho^2} dW_F(t) \equiv \rho dW_\sigma(t) + \overline{\rho} dW_F(t) \tag{10}$$

where $< dW_F, dW_\sigma >= 0$. Equation (9) can be written as

$$F(T,T) = F(0,T) \exp\left(-\frac{1}{2}\int_0^T \sigma(t)^2 dt + \rho \int_0^T \sigma(t) dW_\sigma(t) + \bar{\rho} \int_0^T \sigma(t) dW_F(t) \right) \tag{11}$$

Therefore, given a volatility path $\boldsymbol{\sigma}(0,T)$, the asset $F(T,T)$ is conditionally lognormal and can be re-expressed as

$$F(T,T)|\boldsymbol{\sigma}(0,T) = F(0,T)\exp\left(-\frac{1}{2}\vartheta(T)T + \rho\int_0^T \sigma(t)dW_\sigma(t)\right)\exp\left(\bar{\rho}\int_0^T \sigma(t)dW_F(t)\right) \quad (12)$$

where

$$\vartheta = \frac{1}{T}\int_0^T \sigma(t)^2 dt \quad (13)$$

is the mean integrated variance. The residual stochasticity following conditioning is within the second exponential. The conditional asset is lognormal with

$$\int_0^T \sigma(t)dW_F(t) = \sqrt{\vartheta(T)}W'_F(T) \quad (14)$$

in distribution, giving the conditional lognormal variance as $(1-\rho^2)\vartheta(T)$ and the conditional forward rate

$$\mathbb{E}[F(T,T)|\boldsymbol{\sigma}(0,T)] = F(0,T)\exp\left(-\frac{1}{2}\vartheta(T)T + \rho\int_0^T \sigma(t)dW_\sigma(t)\right)\mathbb{E}\left[\exp\left(\bar{\rho}\sqrt{\vartheta(T)}W_F(T)\right)\right] \quad (15)$$

$$= F(0,T)\exp\left(-\frac{1}{2}\rho^2\vartheta(T)T + \rho\int_0^T \sigma(t)dW_\sigma(t)\right) \quad (16)$$

and so allowing for standard Black-Scholes formula to be used to calculate the value of the vanilla option conditional on the volatility path. Hence, given a volatility path,

$$\vartheta(T) = \frac{1}{T}\int_0^T \sigma(t)^2 dt \qquad \text{and} \qquad \mathrm{I}_{\sigma\mathrm{dW}} = \int_0^{\mathrm{T}} \sigma(\mathrm{t})\mathrm{dW}_\sigma(\mathrm{t}) \quad (17)$$

fully characterize the conditional lognormal asset.

The above argument has not assumed so far a specific volatility process except it is driven by a single correlated Wiener process, as in equation (10). For the case of lognormal volatility used in the SABR model,

$$d\sigma(t) = \xi\sigma(t)dW_\sigma(t) \quad (18)$$

the second quantity in equation (17) simplifies further as follows

$$I_{\sigma dW} = \int_0^T \sigma(t)dW_\sigma(t) = \int_0^T \frac{d\sigma(t)}{\xi} = \frac{1}{\xi}(\sigma(T)-\sigma(0)) \quad (19)$$

Therefore the option price can be determined by integrating over the joint density of $\sigma(T)$ (which yields $I_{\sigma dW}$) and $\vartheta(T)$. Calling this density $\psi(\sigma(T),\vartheta(T))$ then a vanilla option $V_{SABR}(F,K,T)$

can be written, using $F_\sigma(T,T) = \mathbb{E}[F(T,T)|\sigma(0,T)]$, as

$$V_{SABR}(F,K,T) = \int_0^\infty d\sigma(T) \int_0^\infty d\vartheta(T)\psi(\sigma(T),\vartheta(T))V_{BS}(F_\sigma(T,T),\vartheta(T),K,T) \tag{20}$$

However, the distribution of $\sigma(T)$ is known, so writing $\psi(\sigma(T),\vartheta(T)) = \psi_m(\sigma(T))\psi_c(\vartheta(T)|\sigma(T))$, then this can be further rewritten as

$$V_{SABR}(F,K,T) = \int_0^\infty d\sigma(T)\psi_m(\sigma(T)) \int_0^\infty d\vartheta(T)\psi_c(\vartheta(T)|\sigma(T))V_{BS}(F_\sigma(T,T),\vartheta(T),K,T) \tag{21}$$

The calculation therefore depends on the ability to calculate the conditional density $\psi_c(\vartheta(T)|\sigma(T))$. This is performed using a lognormal approximation, $\psi^*(\vartheta(T)|\sigma(T)) \approx \psi_c(\vartheta(T)|\sigma(T))$, which is matched to

$$\mathbb{E}\left[\left(\frac{1}{T}\int_0^T \sigma^2(t)dt\right)\middle|\,\sigma(T)\right] \qquad \text{and} \qquad \mathbb{E}\left[\left(\frac{1}{T}\int_0^T \sigma^2(t)dt\right)^2\middle|\,\sigma(T)\right] \tag{22}$$

The instantaneous volatility, conditional on the terminal instantaneous volatility is expressed via the Brownian Bridge as

$$\sigma(t)|\sigma(T) = \sigma(0)\exp\left(-\frac{1}{2}\xi^2 t + \frac{\xi t}{\sqrt{T}}\zeta + \xi\left(1-\frac{t}{T}\right)^{1/2} W_\sigma(t)\right) \tag{23}$$

where

$$\zeta = \frac{\ln\sigma(T) - \ln\sigma(0) + \frac{1}{2}\xi^2 T}{\xi\sqrt{T}} \tag{24}$$

This then yields

$$\mathbb{E}\left[\frac{1}{T}\int_0^T \sigma^2(t)dt\middle|\,\sigma(T)\right] = \frac{\sigma^2(0)}{T}\frac{\sqrt{2\pi T}}{2\xi}\exp\left(\frac{1}{2}q^2\frac{\xi^2}{T}\right)\left[N(\phi(T)) - N(\phi(0))\right] \tag{25}$$

Here $N(...)$ is the cumulative normal function and

$$\phi(t) = \frac{2\xi}{\sqrt{T}}\left(t - \frac{1}{2}q\right) \qquad \text{and} \qquad q = T + \frac{1}{\xi^2}\log\left(\frac{\sigma(T)}{\sigma(0)}\right) \tag{26}$$

Further

$$\mathbb{E}\left[\left(\frac{1}{T}\int_0^T \sigma^2(t)dt\right)^2\middle|\,\sigma(T)\right] = \sigma^2(0)\frac{\sqrt{2\pi T}}{\xi T^2}\int_0^T dt\exp\left(\frac{1}{2}\bar{q}^2\frac{\xi^2}{T^2}\right)\left[N(\phi(T)) - N(\phi(t))\right] \tag{27}$$

where $\bar{q} = q - 4t$ and can be conducted via a simple numerical integration.

To indicate the effectiveness of representing the joint density of $\sigma(T)$ and $\vartheta(T)$ as an integral over lognormal densities, it is worthwhile to considering the ability to capture moments of the mean integrated variance distribution which can be calculated analytically. Defining

$$\sigma_k(T) = \left(\mathbb{E}\left[\left(\frac{1}{T}\int_0^T \sigma^2(t)dt \right)^k \right] \right)^{1/2k} \tag{28}$$

for k = 1,2,3,..., then the expectation can be calculated as

$$\mathbb{E}\left[\left(\frac{1}{T}\int_0^T \sigma^2(t)dt \right)^k \right] = \mathbb{E}\left[\frac{k!}{T}\int_0^T dT_1 \int_{T_1}^T dT_2 ... \int_{T_k-1}^T dT_k \sigma^2(T_1)\sigma^2(T_2)...\sigma^2(T_k) \right] \tag{29}$$

which gives the analytic forms[1]

$$\mathbb{E}\left[\left(\frac{1}{T}\int_0^T \sigma^2(t)dt \right) \right] = \frac{\sigma^2(0)}{\xi^2 T}\left(e^{\xi^2 T} - 1 \right) \tag{33}$$

$$\mathbb{E}\left[\left(\frac{1}{T}\int_0^T \sigma^2(t)dt \right)^2 \right] = \frac{2!\sigma^4(0)}{\xi^4 T^2}\left(\frac{1}{30}e^{6\xi^2 T} - \frac{1}{5}e^{\xi^2 T} + \frac{1}{6} \right) \tag{34}$$

$$\mathbb{E}\left[\left(\frac{1}{T}\int_0^T \sigma^2(t)dt \right)^3 \right] = \frac{3!\sigma^6(0)}{\xi^6 T^3}\left(\frac{1}{1890}e^{15\xi^2 T} - \frac{1}{270}e^{6\xi^2 T} + \frac{1}{70}e^{\xi^2 T} - \frac{1}{90} \right) \tag{35}$$

$$\mathbb{E}\left[\left(\frac{1}{T}\int_0^T \sigma^2(t)dt \right)^4 \right] = \frac{4!\sigma^8(0)}{\xi^8 T^3}\left(\frac{1}{216216}e^{28\xi^2 T} - \frac{1}{24570}e^{15\xi^2 T} + \frac{1}{5940}e^{6\xi^2 T} - \frac{1}{1890}e^{\xi^2 T} + \frac{1}{2520} \right) \tag{36}$$

In McGhee [2011] a comparison between these analytic quantities in equations (33) - (36) and those

[1]Note the first two expectations are presented in Hull and White [1987]. There the instantaneous variance, $V(t) = \sigma^2(t)$ is assumed to follow a lognormal process

$$dV = \mu V dt + \xi V dz \tag{30}$$

for a Wiener process dz. Defining the "mean variance"

$$\bar{V} = \frac{1}{T}\int_0^T \sigma^2(t)dt \tag{31}$$

then it is shown, with $V_0 = \sigma^2(0)$, that

$$\mathbb{E}[\bar{V}] = \frac{e^{\mu T} - 1}{\mu T}V_0; \qquad \mathbb{E}[\bar{V}^2] = \left[\frac{2e^{(2\mu+\xi^2)T}}{(\mu+\xi^2)(2\mu+\xi^2)T^2} + \frac{2}{\mu T^2}\left(\frac{1}{2\mu+\xi^2} - \frac{e^{\mu T}}{\mu+\xi^2} \right) \right] V_0 \tag{32}$$

For the process considered here, $d\sigma = \xi\sigma dW$ and so $dV = \xi^2 V dt + 2\xi V dW'(t)$. Replacing $\mu \to \xi^2$ and $\xi \to 2\xi$ in the above recovers the first two expressions in the text.

derived from the integration scheme above are compared. Explicitly, within the integration scheme

$$\mathbb{E}\left[\left(\frac{1}{T}\int_0^T \sigma^2(t)dt\right)^k\right] \approx \int_0^T d\sigma(T)\psi_m(\sigma(T)) \int_0^T d\vartheta\psi^*(\vartheta|\sigma(T))\vartheta^k \tag{37}$$

Expectations in (22) ensure the analytic form is matched for $k = 1$ and 2. A simpler approximation would be to estimate the distribution of the mean integrated variance as a lognormal, say $\psi_{ln}(\vartheta)$. In this simpler case

$$\mathbb{E}\left[\left(\frac{1}{T}\int_0^T \sigma^2(t)dt\right)^k\right] \approx \int_0^T d\vartheta\psi_{ln}(\vartheta|\sigma(T))\vartheta^k \tag{38}$$

Again the first two expectations will be matched but not the higher order ones. These results are summarized in Table 1 which shows the degree of deviation from lognormality in the distribution of the mean integrated variance, especially for longer maturities, and the accuracy of the integration scheme at capturing the higher order moments.

T	$\sigma(0)$	ξ	k	$\sigma_k(T)$: Analytic	$\sigma_k(T)$: Integration (ψ^*)	$\sigma_k(T)$: Lognormal (ψ_{ln})
2Y	20%	60%	1	24.20%	24.20%	24.20%
2Y	20%	60%	2	34.38%	34.38%	34.38%
2Y	20%	60%	3	54.57%	54.34%	48.82%
2Y	20%	60%	4	93.84%	93.01%	69.34%
5Y	20%	50%	1	28.23%	28.23%	28.23%
5Y	20%	50%	2	59.14%	59.14%	59.14%
5Y	20%	50%	3	156.08%	154.92%	123.90%
5Y	20%	50%	4	455.28%	446.85%	259.58%
30Y	20%	23%	1	31.31%	31.31%	31.31%
30Y	20%	23%	2	87.17%	87.17%	87.17%
30Y	20%	23%	3	321.68%	318.80%	242.70%
30Y	20%	23%	4	1314.30%	1239.95%	675.71%

Table 1: Moments of the Mean Integrated Variance distribution - comparing the analytic, integration and lognormal approximation cases.

3.3 The SABR Finite Difference Scheme

Now consideration will be given to the SABR Process finite difference implementation. Rewriting (3) in terms of $X = \ln F(t,T)$ and $Y = \ln \sigma(t)$,

$$dX(t) = -\frac{1}{2}\sigma(t)^2 dt + \sigma(t)dW_X(t) \qquad \text{and} \qquad d\ln Y(t) = -\frac{1}{2}\xi^2 dt + \xi dW_Y(t) \tag{39}$$

then the corresponding partial differential equation can be written

$$\frac{\partial V}{\partial t} - \frac{1}{2}\sigma^2\frac{\partial V}{\partial X} + \frac{1}{2}\sigma^2\frac{\partial^2 V}{\partial X^2} - \frac{1}{2}\xi^2\frac{\partial V}{\partial Y} + \frac{1}{2}\xi^2\frac{\partial^2 V}{\partial Y^2} + \rho\sigma\xi\frac{\partial^2 V}{\partial X \partial Y} = 0 \tag{40}$$

The scheme (specifically Craig and Sneyd [1988]), for a given set of input parameters, will be used to calculate the value of a vanilla option and from this value the implied volatility determined. This combination of input parameters and output volatility will be used, as before, to train a neural network.

Given that the SABR Approximation and the SABR Process (either under the integration or finite difference implementation) are closely related, the starting point of the calibration process for the SABR Process volatilities will be the network that has been trained for the SABR Approximation. Within the machine learning literature, this is referred to as "transfer learning" (Pratt et al. [1991]).

This is an important principle: for complex problems, a fast approximation, which captures dominant structural features can be used to pre-train a network whilst a more computationally expensive and accurate dataset is being compiled.

4 Methodology

4.1 The Artificial Neural Network

It will be assumed that the reader has an elementary understanding of simple Neural Networks. Detailed texts on Neural Networks and Machine Learning include Bishop [1996] and Bishop [2011].

The neural networks tested in this article are constructed with one hidden layer only but with differing numbers of nodes, specifically 250, 500, 750 and 1,000. The optimizer used is "Adam" (see Kingma and Ba [2014]) with a loss function of "mean squared error". The activation function used on the input nodes and the hidden layer is the "softplus" function

$$f(x) = \ln(e^x + 1) \tag{41}$$

The softplus activation function is a smooth version of the commonly used Rectified Linear Unit, "ReLU" function

$$f(x) = \max(x, 0) \tag{42}$$

An adaptive learning rate is used by adjusting the rate of learning depending on the performance during an epoch. For each epoch the learning rate will be gradually decreased. However, when the loss function during the epoch plateaus, the learning rate will be increased. This allows the calibration procedure to dynamically adjust through time.

The aim is to calibrate a neural network to a wide parameter range, specifically the initial volatility, volatility of volatility and correlation, as well as across multiple maturities and strikes. Sampling is taken from the relevant range, both for training and for validation. The chosen parameter ranges for time (1 day to 2 years), initial volatility (5% to 50%) or correlation (-90% to +90%). However, fixed levels are not deemed appropriate for the volatility of volatility and the strike. In the case of volatility of volatility, this is typically very high at short tenors decaying to longer tenors. The heuristic employed is

$$\xi_t = \xi_s \times \sqrt{\frac{T_s}{T_t}}, \tag{43}$$

so that for $T_t > T_s$, $\xi_t < \xi_s$. Fixing a range for the ξ parameter at $T_s = 1$ month (say) as 5% to 400% then leads to an appropriate range for other expiries, as shown in Table 2.

Tenor	1W	1M	3M	6M	1Y	2Y
Maximum	833%	400%	231%	163%	115%	82%
Minimum	10%	5%	3%	2%	1%	1%

Table 2: Term structure range for volatility of volatility parameter.

Sampling an appropriate strike range dynamically as a function of maturity and SABR parameters is more challenging. Given a η_σ standard deviation move in the terminal instantaneous volatility for a given expiry T,

$$\sigma(T, \eta_\sigma) = \sigma(0) \exp\left(-\frac{1}{2}\xi^2 T + \xi\eta_\sigma\sqrt{T}\right) \tag{44}$$

this will feed into the location of the density distribution via the correlation term. For the analysis performed in this paper, the choice of $\eta_\sigma = 1.5$ is made. The proposed limits of the strike range, for $\pm\eta_S$, are

$$\begin{aligned} K(\pm\eta_S, \eta_\sigma) &= F(0,T)\exp\left(-\frac{1}{2}\mathbb{E}\left[\frac{1}{T}\int_0^T \sigma^2(s)ds\right]T \right. \\ &\qquad \left. +\frac{\rho}{\xi}(\sigma(T,\eta_\sigma)-\sigma(0)) \pm \eta_S\sqrt{\mathbb{E}\left[\frac{1}{T}\int_0^T \sigma^2(s)ds\right]T}\right) \qquad (45) \\ &= F(0,T)\exp\left(-\frac{1}{2}\frac{\sigma^2(0)}{\xi^2}\left(e^{\xi^2 T}-1\right)\right. \\ &\qquad \left. +\frac{\rho}{\xi}(\sigma(T,\eta_\sigma)-\sigma(0)) \pm \eta_S\frac{\sigma(0)}{\xi}\left(e^{\xi^2 T}-1\right)^{1/2}\right) \qquad (46) \end{aligned}$$

Strike limits deeper into the tails of the distribution can be chosen if required but rely on the ability

to construct accurate implied volatilities in this region.

Table 3, with $\sigma_0 = 20\%$, $\rho = -50\%$ and a 1 month volatility of volatility of 100%, shows a range of upper and lower strikes across different maturities (using equation (36) to determine the volatility of volatility level). For the same case but with $\rho = +50\%$, is shown in Table 4 where the strikes due to the change in correlation are moved higher.

T	2W	1M	2M	3M	6M	9M	1Y
ξ	147.2%	100%	70.7%	57.7%	40.8%	33.3%	28.9%
$K(\eta_S = +3.5, \eta_\sigma = 1.5)$	1.1128	1.1697	1.2468	1.3090	1.4589	1.5837	1.6956
$K(\eta_S = -3.5, \eta_\sigma = 1.5)$	0.8407	0.7741	0.6956	0.6404	0.5309	0.4591	0.4059

Table 3: Strike range for $\sigma_0 = 20\%$ and $\rho = -50\%$

T	2W	1M	2M	3M	6M	9M	1Y
ξ	147.2%	100%	70.7%	57.7%	40.8%	33.3%	28.9%
$K(\eta_S = +3.5, \eta_\sigma = 1.5)$	1.1876	1.2873	1.4277	1.5452	1.8448	2.1109	2.3629
$K(\eta_S = -3.5, \eta_\sigma = 1.5)$	0.8972	0.8520	0.7965	0.7560	0.6713	0.6120	0.5657

Table 4: Strike range for $\sigma_0 = 20\%$ and $\rho = +50\%$

For given upper and lower strike limits, $K_\pm$, ten strikes were chosen by uniformly dividing $\eta_S \in [-3.5, +3.5]$. The intention is to present the network with a range of sample points that represent the functional form of the smile. The resulting inputs to the network are $\{T, \sigma_0, \xi, \rho, K_1 = K_-, ..., K_{10} = K_+\}$ which give rise to ten volatility outputs $\{\sigma_1, ..., \sigma_{10}\}$. Figure 1 shows a schematic representation of the ANN.

A further refinement is if a logical division exists in the parameter space then a different neural network can be calibrated to each. This is done here in the maturity dimension with an ANN calibrated to the model for $T \in [1D, 1Y]$ and a second network calibrated to $T \in [1Y, 2Y]$. Additional neural networks can be added for later time periods.

The network is trained on 250,000 input-output pairs and validated on a test set of 50,000 pairs. The inputs are chosen both deterministically from the domain space as well as randomly. The time domain is split into 100 equidistant points, σ_0, ξ and ρ are split into 10 equidistant points, giving rise to 100x10x10x10 = 100,000 selected points from the input space. A further 150,000 points are randomly chosen giving rise to the 250,000 training samples. The remaining 50,000 test points are randomly chosen also.

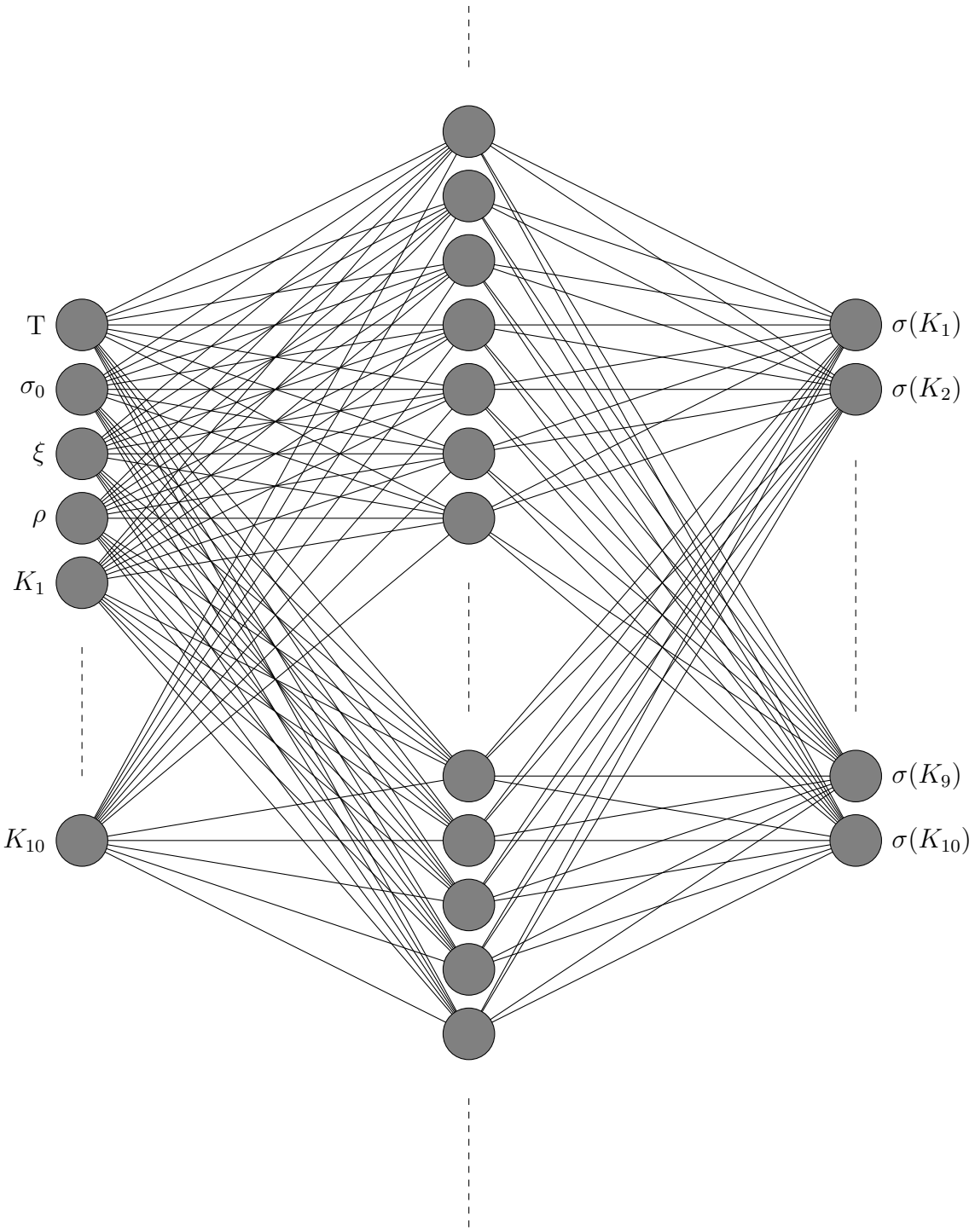


Figure 1: A schematic representation of the neural network used with one hidden layer. At each node the activation function is applied.

4.2 Hardware and Software

The neural network calculations are performed using the Python 3.6.5, Keras 2.2.2 and Tensorflow 1.10.0 libraries. Calculations were split across an Intel Core i7-7700HQ CPU @ 2.80Ghz machine with 4 cores and 8 logical processors and an Intel Core i7-3770 CPU @ 3.40GHz machine with 4 cores and 8 logical processors. The latter machine also had a NVIDIA GeForce GTX 660 GPU. Tensorflow ANN learning algorithms were performed on both CPUs and the single GPU.

4.3 Smile Construction, Cumulative Distribution Function and Probability Density Function

For the neural network to be of practical use, smooth first and second derivatives are required which allow for the calculation of the cumulative distribution function (CDF) and probability density function (PDF). This is achieved, not by directly differentiating the neural network with respect to its inputs but by using a technique common to other numerical schemes such as finite difference schemes. For a given parameter set, including strikes, the neural network will return the corresponding volatilities. These are then used to construct a cubic spline which is in turn used for both volatility interpolation and in the calculation of first and second derivatives. Smile functions and both CDFs and PDFs are shown in the results section.

5 Test Results

5.1 Smiles under the ANN for the SABR Approximation

Here a number of sample smiles are produced for the ANN calibrated to the SABR Approximation. The network calibrated to $T \in [1D, 1Y]$ is considered to demonstrate the range of smiles it can produce across the parameter space. The technique exhibits a high degree of accuracy when compared to the original formula as well as that of the cumulative distribution and implied probability density functions. Indeed, in the graphs shown below it is difficult to distinguish between the original function and those corresponding to the four different networks. As such the errors of each network with respect to the target function are also shown.

Firstly, three examples of different smiles are considered. Figure 2 shows a 2-week smile with initial volatility 30%, volatility of volatility 160% and a correlation of -60% with an error in the range $[-0.03\%, +0.10\%]$. In figure 3, a 6-month smile with initial volatility 30%, volatility of volatility 40% and correlation of 0% is shown. Here the error is within $[-0.015\%, +0.02\%]$. Finally, figure 4 shows a 1-year smile with initial volatility 20%, volatility of volatility 30% and a correlation of $+30\%$. The error in this case is within $[-0.025\%, 0.02\%]$.

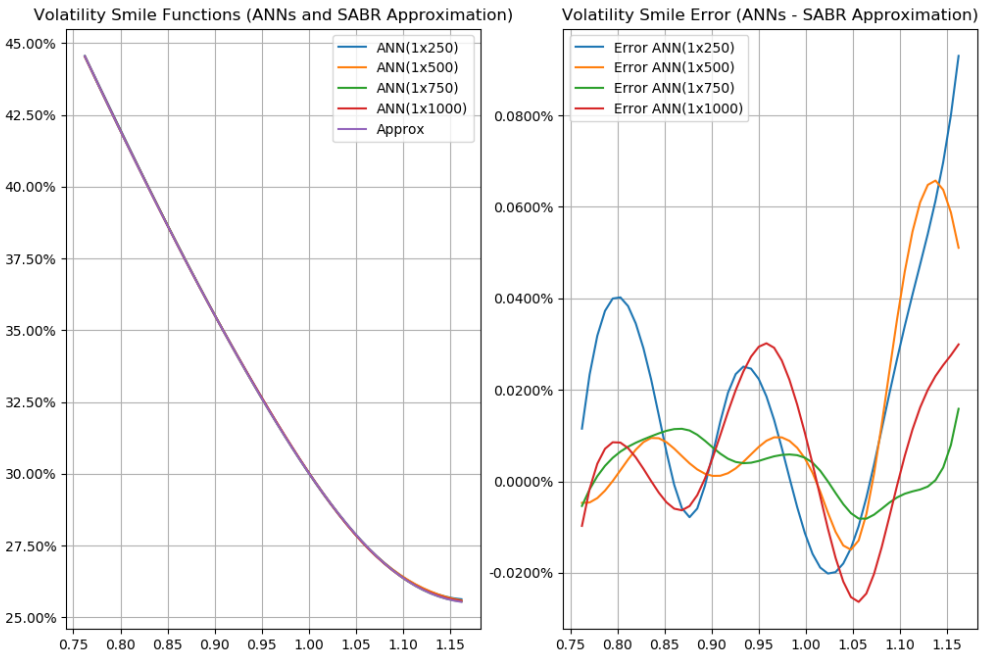


Figure 2: ANN Smiles and SABR Approximation Comparison ($T = 14d$, $\sigma_0 = 30\%$, $\xi = 160\%$, $\rho = -60\%$)

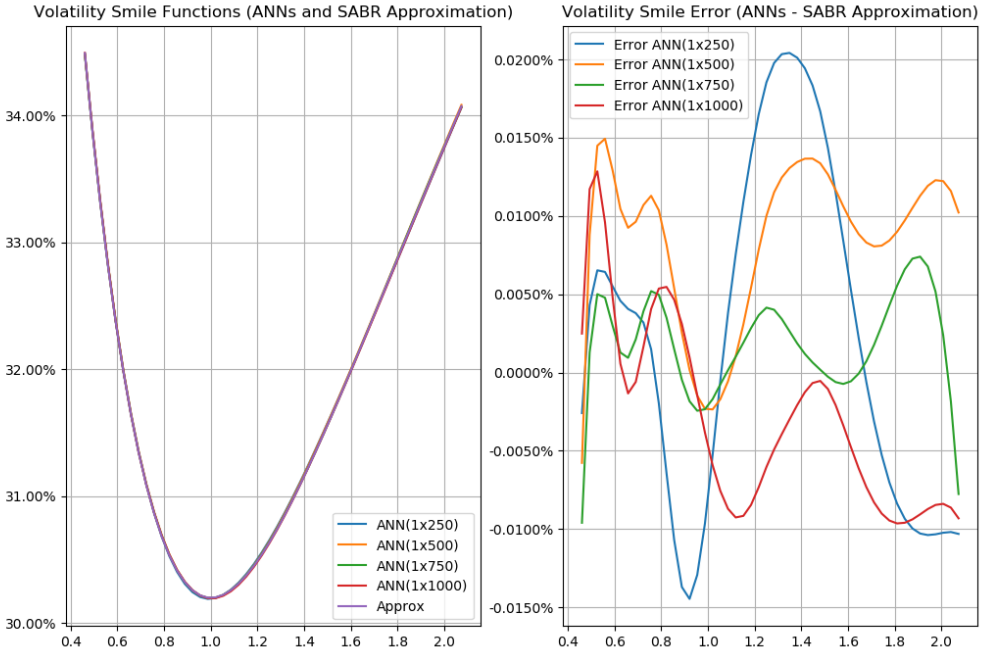


Figure 3: ANN Smiles and SABR Approximation Comparison ($T = 6M$, $\sigma_0 = 30\%$, $\xi = 40\%$, $\rho = 0\%$)

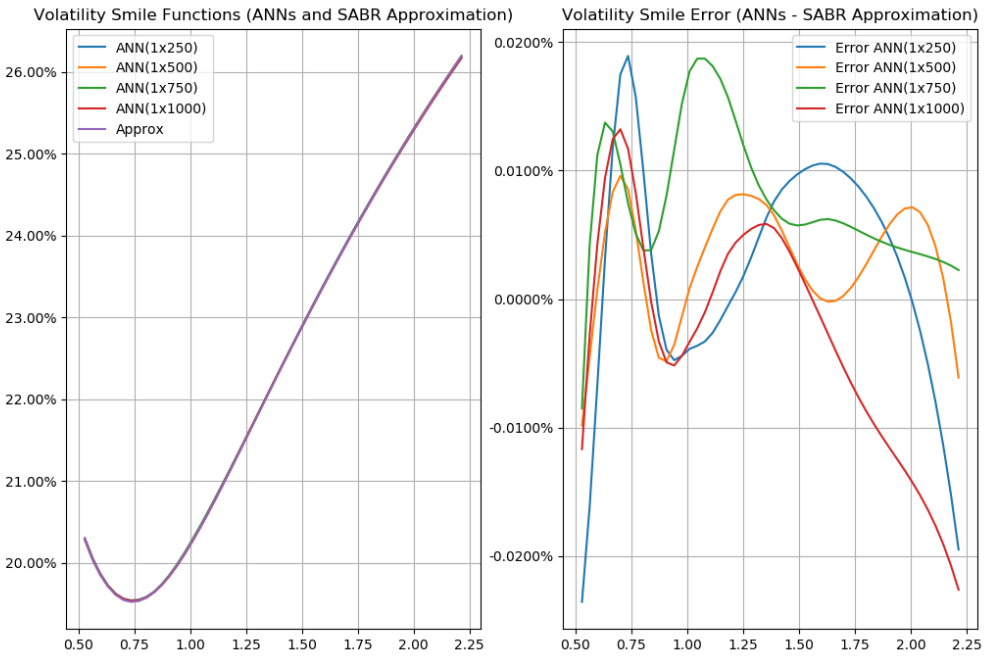


Figure 4: ANN Smiles and SABR Approximation Comparison ($T = 1Y$, $\sigma_0 = 20\%$, $\xi = 30\%$, $\rho = +30\%$)

Figures 5 and 6 examine, in addition to the volatility smile functions, both the CDFs and PDFs. In both cases the network with 1,000 nodes in the hidden layer is used. Considering firstly Figure 5, a one-month maturity is considered with initial volatility 30%, volatility of volatility 150% and a correlation of -75%. The smile function itself is to within approximately $[-0.03\%, +0.02\%]$. Both the CDF and PDF are smooth functions due to the cubic spline interpolation. In terms of accuracy, the CDF is within $[-0.06\%, +0.04\%]$. The PDF at its worse has error of 2% in 500%, a relative error of 0.4%. Figure 5 takes a longer maturity of nine months, initial volatility 20%, volatility of volatility 30% and a positive correlation of $\rho = +50\%$. As before, the smile function, CDF and PDF are smooth with small errors. For the smile function it is within $[-0.0025\%, +0.0150\%]$. The CDF has error in the range $[-0.02\%, +0.03\%]$ and the PDF has a maximum error of approximately 0.4% in 250%, or 0.16% in relative terms.

These results show the robustness of the method in accurately capturing the target function and the ability to construct smooth first and second derivatives. Attention now turns to the more computationally expensive schemes.

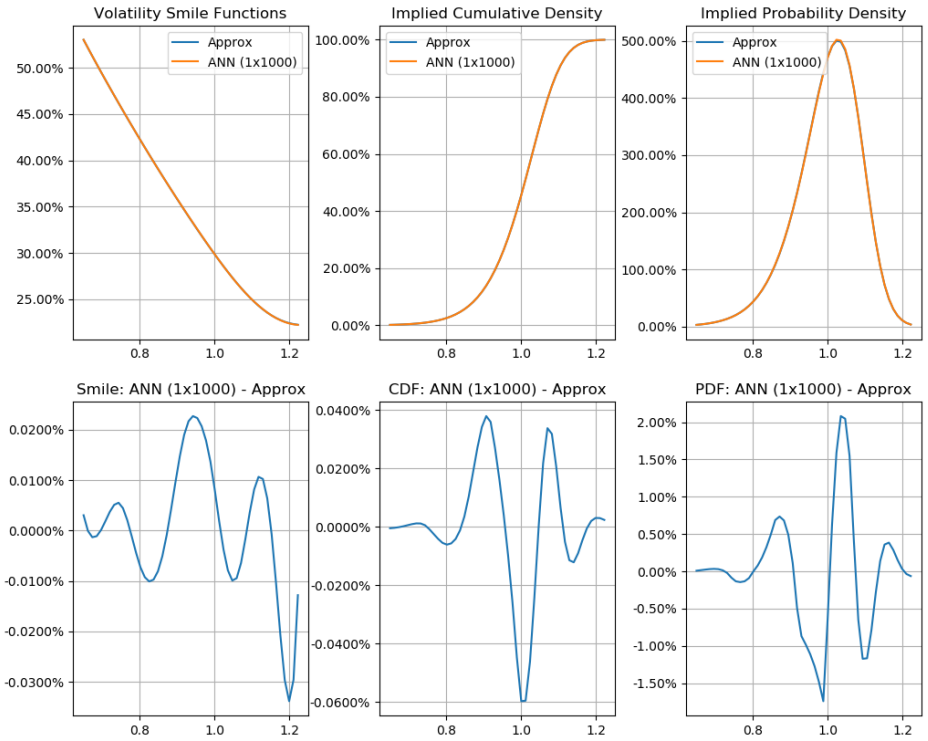


Figure 5: Smiles, CDFs and PDFs ($T = 1M$, $\sigma_0 = 30\%$, $\xi = 150\%$, $\rho = -75\%$)

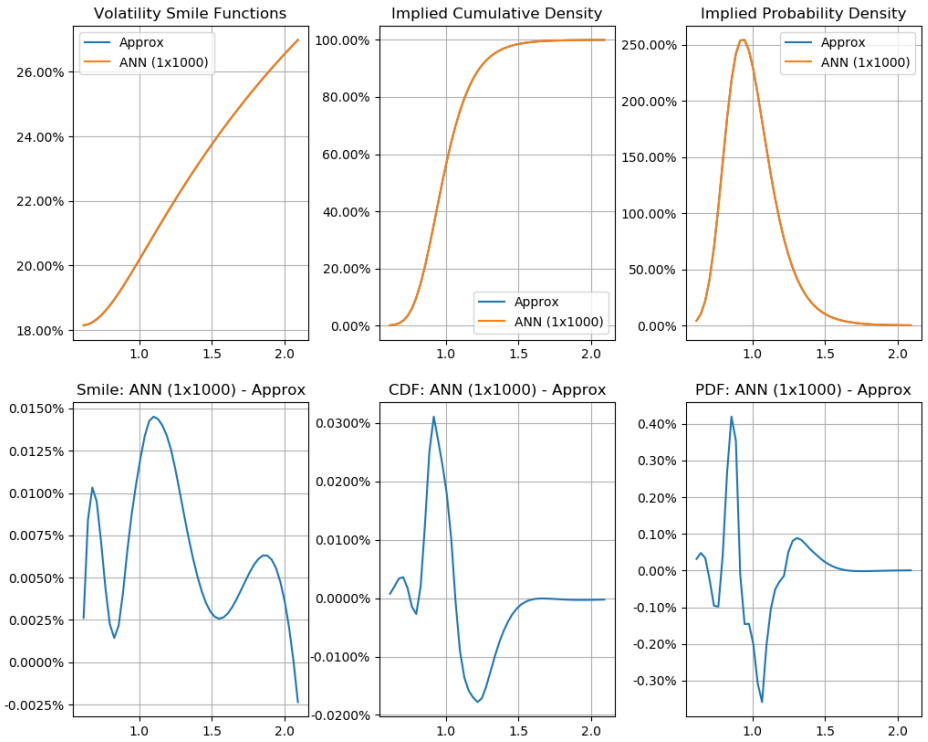


Figure 6: Smiles, CDFs and PDFs ($T = 9M$, $\sigma_0 = 20\%$, $\xi = 30\%$, $\rho = +50\%$)

5.2 Comparing SABR Process and SABR Approximation Smiles

Before examining the ANN calibrated to one of the more accurate numerical scheme, the degree to which the SABR Approximation smile deviates from the SABR Process smile (generated from both the SABR Integration scheme or by the SABR 2FPDE) will be investigated. Considering the 10 generated strikes for a given smile, the degree of error will be given by mean absolute error (MAE) *i.e.*

$$\text{MeanAbsError} = \frac{1}{10}\sum_{\text{i}=1}^{10} |\text{PV}(\sigma_{\text{approx}}, \text{K}_\text{i}) - \text{PV}(\sigma_{\text{process}}, \text{K}_\text{i})| \tag{47}$$

Within the population of 300,000 smile curves generated for $T \in [1Y, 2Y]$, the mean absolute errors between the SABR approximation and the SABR integration scheme are shown in Table 5. Less than half of the sample space have smiles with a mean absolute error, in price terms, less than $1bp$, with almost 25% having MAE in excess of $10bps$.

MeanAbsError	$\leq 1bp$	$\leq 2bp$	$\leq 5bp$	$\leq 10bp$	$\leq 20bp$	$\leq 50bp$	$\leq 100bp$	$\leq 150bp$	$\leq 200bp$
Population %	39.13%	47.99%	61.89%	73.41%	83.61%	93.47%	97.79%	99.10%	99.63%

Table 5: Degree of deviation between SABR Approximation and SABR Integration schemes

Strike	Approximation	Error vs 2FPDE	Integration	Error vs 2FPDE	2FPDE
0.4419	33.47%	+3.45%	29.97%	−0.05%	30.02%
0.5152	29.66%	+2.88%	26.72%	−0.06%	26.78%
0.6007	25.70%	+2.31%	23.33%	−0.06%	23.39%
0.7004	21.54%	+1.73%	19.76%	−0.05%	19.81%
0.8166	17.14%	+1.09%	16.00%	−0.05%	16.05%
0.9520	12.64%	+0.42%	12.20%	−0.03%	12.23%
1.1100	10.49%	−0.04%	10.51%	−0.02%	10.53%
1.2941	12.82%	+0.27%	12.54%	−0.01%	12.55%
1.5088	16.00%	+0.77%	15.24%	+0.01%	15.23%
1.7591	19.19%	+1.26%	17.94%	+0.01%	17.93%
2.0509	22.31%	+1.74%	20.56%	+0.00%	20.56%

Table 6: Implied Volatilities comparing SABR Approximation, Integration and 2FPDE Methods for $T = 1.6316$, 10.8266%, 88.7903% and $\rho = -46.4053\%$

Strike	Approximation	Error vs 2FPDE	Integration	Error vs 2FPDE	2FPDE
0.4419	0.30%	+0.15%	0.15%	+0.00%	0.15%
0.5152	0.43%	+0.19%	0.24%	+0.00%	0.24%
0.6007	0.65%	+0.24%	0.41%	−0.01%	0.41%
0.7004	1.05%	+0.29%	0.75%	−0.01%	0.76%
0.8166	1.89%	+0.32%	1.56%	−0.01%	1.57%
0.9520	4.17%	+0.20%	3.96%	−0.01%	3.97%
1.1100	1.76%	−0.01%	1.77%	−0.01%	1.77%
1.2941	0.46%	+0.04%	0.41%	+0.00%	0.41%
1.5088	0.20%	+0.06%	0.15%	+0.00%	0.15%
1.7591	0.12%	+0.05%	0.07%	+0.00%	0.07%
2.0509	0.08%	+0.04%	0.04%	+0.00%	0.04%

Table 7: OTM Option Prices comparing SABR Approximation, Integration and 2FPDE Methods for $T = 1.6316$, 10.8266%, 88.7903% and $\rho = -46.4053\%$

Table 7 shows the different methods in price terms for out-the-money (OTM) options. Importantly, the SABR Integration and SABR 2FPDE show agreement to within $1bp$. However, the SABR approximation in this case is up to $32bps$ different from the 2FPDE.

5.3 The ANN for the SABR Process (Integration and 2FPDE)

It remains to consider the ANN calibrated to one of the more accurate schemes - here the SABR Integration is chosen. As will be detailed in the next section, the SABR Integration and SABR 2FPDE are time consuming to produce. To generate the 3 million volatilities for the training and test datasets takes of the order of 3 hours on one CPU for the integration scheme and of the order of 475 hours for the two factor finite difference scheme. In the finite difference scheme the dataset is distributed across multiple CPUs. Once these datasets are produced, the ANN calibrated to the SABR Approximation can be used as starting point for the ANN trained to another scheme, and so reduce the subsequent training time.

An example is show in Figure 7 for the case of $T = 540$ days, $\sigma(0) = 30\%$, $\xi = 60\%$ and $\rho = -35\%$. The graphs are plotted on a log-scale on the x-axis two draw out the difference between the SABR Approximation and the more accurate schemes. The ANN, SABR Integration and SABR 2FPDE schemes agree to a degree of accuracy. Once calibrated, however, the ANN constructs volatility smiles significantly faster.

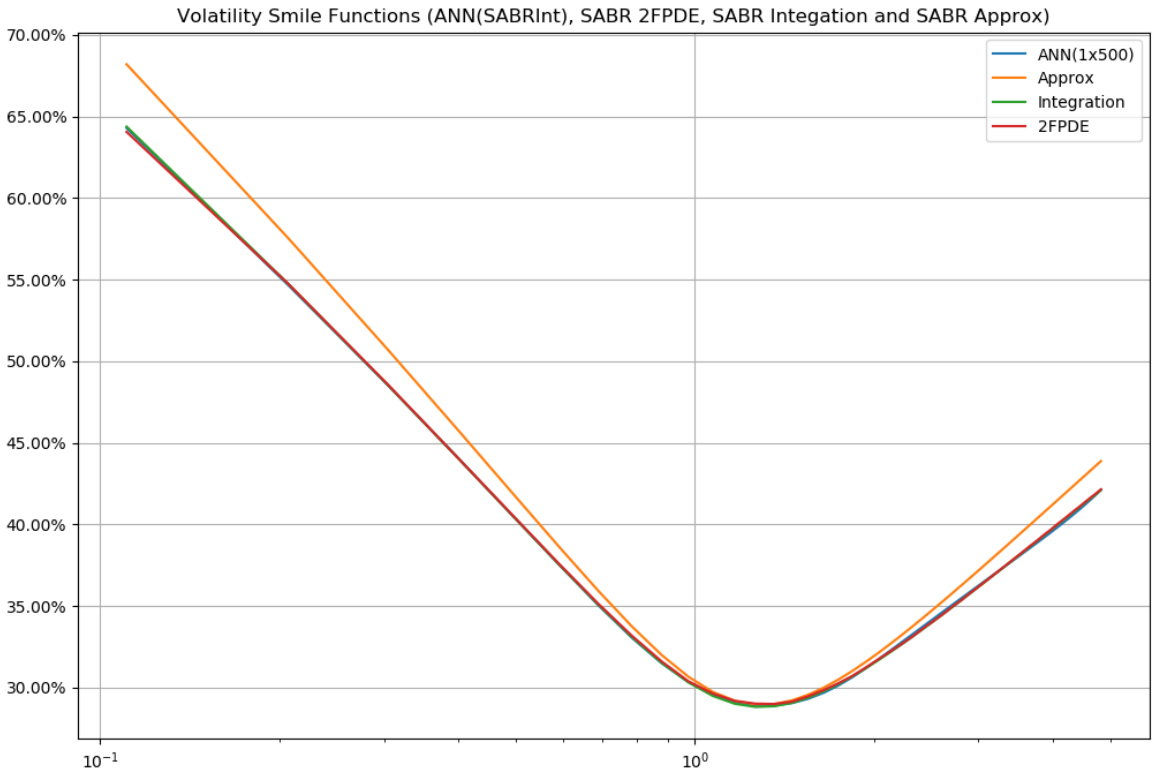


Figure 7: Comparing the SABR Approximation, SABR Integration, SABR 2FPDE and an ANN trained on the SABR Integration scheme for $T = 540d$, $\sigma(0) = 30\%$, $\xi = 60\%$ and $\rho = -35\%$.

5.4 Performance

The objective of this article is to explore the ability of an ANN to represent, to a given tolerance level, an accurate SABR model which does not suffer the weaknesses of the widely used approximation scheme but which is performant enough to be used in practice.

The two schemes used for comparison are the SABR Integration scheme and the SABR two factor finite difference scheme. The neural network calculation, with one hidden layer, involves two matrix multiplication and the application of the activation function across the input nodes and hidden layer.

In the tests, a smile consisting of the 10 sample strikes is constructed. The SABR Integration and SABR 2FPDE are implemented in C++; the ANN is TensorFlow based, accessed via Keras in Python.

Method	Number of Smiles	Time	Time per Smile	Time per Volatility
2FPDE (400x200x100)	100	570s	5.68s	$5.7 \times 10^{-1} s$
Integration	100	3.6s	0.03618	$3.6 \times 10^{-3} s$
ANN	10,000	5.4s	0.00054	$5.4 \times 10^{-5} s$

Table 8: Comparing the performance of the ANN relative to the SABR Integration and SABR 2FPDE schemes.

The ANN performs 65+ times faster than the numerical integration scheme and an impressive 10,000+ times faster than the 2FPDE scheme.

6 Conclusions

There are many examples across Quantitative Finance of complex calculations that need, due to the time-critical nature of the trading environment, to be performed either using accurate approximations or employ a large scale computational infrastructure. In this article, it has been shown that a property of Artificial Neural Networks, the Universal Approximation Theorem, can be used to encapsulate a complex model to a given degree of accuracy. This principle can be applied to a wide range of problems and so place ANNs, trained on a large sample of the model input-output space, as a working alternative to the underlying model.

In the case of approximation schemes, there will be a region of applicability outside of which the method will deteriorate. This is a common problem with the SABR Approximation. As has been demonstrated, with a sufficiently large generated training set, a range of ANNs will perform robustly to a high degree of accuracy in a fraction of the time taken for existing accurate schemes.

In addition to matching the implied volatility from different model implementations, the first derivative (as expressed via the implied cumulative density function) and the second derivative (as expressed via the implied probability density function) can be calculated constructing a spline representation of the volatility smile through a range of datapoints, ensuring stability of the solution.

Against the most accurate scheme investigated, the two factor finite difference scheme, the ANN representation shows a dramatic 10,000 times performance improvement. This demonstrates the effectiveness of adopting ANN representations in place of time consuming complex function calculations across the quantitative finance space. The calculation intensity is shifted from in-situ to an upfront cost. In addition, ANNs can undergo ongoing training to improve and explore a wider domain space than the original training set.

Finally, although the principle of the Universal Approximation Scheme applies across as wide a domain space as is of interest, a more pragmatic approach can be to segregate this space into appropriate subsets. In the case of volatility surface interpolation, splitting by different forward

periods eg. $ANN_1(1D-1Y)$, $ANN_2(1Y-2Y)$, etc. allows for the entire surface to be trained in parallel with smaller and simpler networks than would be the case using one large network to cover it in its entirety.

Looking forward, the work outlined in this paper has a number of future research avenues. A range of networks with a single hidden layer of different sizes was considered. The efficacy of using deep networks (many hidden layers) is worthy of exploration as well as a study of different activation functions and learning algorithms. In addition, the ability to accelerate training using other known information as inputs in the training set warrants consideration.

In terms of further problems to consider, the landscape is wide. The networks used here were trained on the implied volatility function, either obtained directly from the SABR Approximation scheme or as calculated from the pricing of the underlying vanilla options using either the integration or two factor finite difference scheme. The networks could equally have been trained on the vanilla contract values themselves. Indeed, this has been considered for simple products under Black-Scholes dynamics (Brostrom and Kristiansson [2018], Ferguson and Green [2018]). Here stochastic volatility dynamics are considered. This leads to the possibility of pricing more complex products (such as binary and barrier contracts) in the presence of stochastic volatility as ANN representations.

Turning to the underlying stochastic volatility model itself, consideration in this paper have been given to the SABR model. However, a number of extensions exist, such as ZABR (Andreasen and Huge [2013]), stochastic volatility of volatility (McGhee [2013]), mean reverting stochastic volatility, etc. The use of ANN representations make contract valuation under these models e.g. CMS, spread options and a range of exotic contracts more viable for use within a trading environment.

Acknowledgements The author would like to thank Romano Trabalzini, Colin Walsh, Paul McCaffrey, Christopher Potter, Vladimir Piterbarg and Katia Babbar for valuable comments made during the preparation of this paper; and Hugo Hemingway-McGhee for assisting in the hardware set-up and computational parallelization that made many of the calculations feasible.

References

J. Andreasen and B. N. Huge. "Expanded forward volatility". *RISK Magazine*, January 2013.

A. Antonov and M. Spector. "SABR spreads its wings". *RISK Magazine*, July 2013.

C.M. Bishop. *"Neural Networks for Pattern Recognition"*. Clarendon Press, 1996.

C.M. Bishop. *"Pattern Recognition and Machine Learning"*. Springer, 2011.

A. Brostrom and R. Kristiansson. "Exotic Derivatives and Deep Learning". *KTH School of Engineering Sciences report*, 2018.

B. Chen, C. W. Oosterlee, and H. Van Der Weide. "A low-bias simulation scheme for the SABR stochastic volatility model". *International Journal of Theoretical and Applied Finance, Volume 15, no. 2*, 2012.

J. Cox. "Notes on Option Pricing I: Constant Elasticity of Diffusions". *Stanford University (Unpublished draft)*, 1975.

I. J. D. Craig and A. D. Sneyd. "An Alternating-Direction Implicit Scheme for Parabolic Equations with Mixed Derivatives". *Computers & Mathematics with Applications*, 16(4):341–350, 1988.

G. Cybenko. "Approximations by superpositions of sigmoidal functions". *Mathematics of Control, Signals, and Systems*, 2(4):303–314, 1989.

R. Ferguson and A. D. Green. "Deeply Learning Derivatives". *http://ssrn.com/abstract=3244821*, 2018.

P. S. Hagan, D. Kumar, A. S. Lesniewski, and D. E. Woodward. "Managing Smile Risk". *Wilmott Magazine*, pages 84–108, September 2002.

K. Hornik. "Approximation Capabilities of Multilayer Feedforward Networks". *Neural Networks*, 4:251–257, 1991.

J. Hull and A. White. "The Pricing of Options on assets with Stochastic Volatility". *The Journal of Finance*, 42(2):281–300, 1987.

O. Islah. "Solving SABR in Exact Form and Unifying it with LIBOR Market Model". *http://ssrn.com/abstract=1489428*, 2009.

D. P. Kingma and J. Ba. "Adam: A Method for Stochastic Optimization". *Proceedings of the 3rd International Conference on Learning Representations (ICLR)*, 2014.

W. A. McGhee. "An Efficient Implementation of Stochastic Volatility by the Method of Conditional Integration". ICBI Global Derivatives and Risk Management, April 2011.

W. A. McGhee. "Decoding the Volatility Smile". *ICBI Global Derivatives & Risk Management Conference, Amsterdam.*, 2013.

W. A. McGhee and R. Trabalzini. "Deconstructing the Volatility Smile". *Available at SSRN: https://ssrn/abstract=2471052*, 2014.

L. Y. Pratt, J. Mostow, and C. A. Kamm. "Direct transfer of learned information among neural networks". *Proceedings of the Ninth National Conference on Artificial Intelligence (AAAI-91), pp584-589, Anaheim, CA*, 1991.